Status	Finished
Started	Friday, 30 October 2026, 3:29 PM
Completed	Friday, 30 October 2026, 3:34 PM
Duration	4 mins 30 secs
Grade	**14.00** out of 15.00 (**93.33**%)

Question **1**

Incorrect

Mark 0.00 out of 1.00

What happens in a binary search if the list has an even number of elements?

a. The higher middle element is chosen as the middle element
b. The lower middle element is chosen as the middle element ✗
c. The middle element is chosen randomly
d. The search stops

Question **2**

Correct

Mark 1.00 out of 1.00

Which of the following is a limitation of binary search?

a. It can only be applied to large lists
b. It is slower than linear search for small lists
c. It does not work with negative numbers
d. It requires the list to be sorted ✓

Question **3**

Correct

Mark 1.00 out of 1.00

In linear search, if the target element is not found in the list, what is the result?

a. The first element is returned
b. The last element is returned
c. The search is considered unsuccessful ✓
d. An error is raised

Question **4**

Correct

Mark 1.00 out of 1.00

What is the key characteristic of binary search?

- a. It can be applied only if the list is sorted ✓
- b. It always starts from the beginning of the list
- c. It compares elements sequentially
- d. It works on unsorted lists

Question **5**

Correct

Mark 1.00 out of 1.00

In binary search, if the target element is less than the middle element, where does the search continue?

- a. In the left sublist ✓
- b. In the entire list
- c. At the beginning of the list
- d. In the right sublist

Question **6**

Correct

Mark 1.00 out of 1.00

In which situation is linear search more efficient than binary search?

- a. When the list is small and unsorted ✓
- b. When the list is large and unsorted
- c. When the list is large and sorted
- d. When the list is small and sorted

Question **7**

Correct

Mark 1.00 out of 1.00

What is the best-case time complexity of linear search?

- a. O(1) ✓
- b. O(log n)
- c. O👎
- d. O(n log n)

Question **8**

Correct

Mark 1.00 out of 1.00

Which of the following is not a limitation of binary search algorithm?

a. Must use a sorted array

b. Requirement of sorted array is expensive when a lot of insertion and deletions are needed

c. There must be a mechanism to access middle element directly

d. Binary search algorithm is not efficient when the data elements more than 1500 ✓

Question **9**

Correct

Mark 1.00 out of 1.00

If a list contains 1000 elements, how many comparisons would a binary search typically make in the worst case?

a. 100

b. 10 ✓

c. 500

d. 1000

Question **10**

Correct

Mark 1.00 out of 1.00

Which method of searching involves sequentially comparing each element until a match is found?

a. Jump search

b. Linear search ✓

c. Binary search

d. Hashing

Question **11**

Correct

Mark 1.00 out of 1.00

Which of the following scenarios is best suited for applying binary search?

a. When the list is sorted ✓

b. When the list contains duplicate elements

c. When the list is very small

d. When the list is unsorted

Question **12**

Correct

Mark 1.00 out of 1.00

What is searching in the context of computer science?

a. Deleting elements from a list
b. Determining whether an element is present in a list ✓
c. Sorting elements in a list
d. Inserting elements into a list

Question **13**

Correct

Mark 1.00 out of 1.00

In a linear search, how many comparisons are made in the worst-case scenario to find an element in a list of size n?

a. 1
b. n/2
c. log n
d. n ✓

Question **14**

Correct

Mark 1.00 out of 1.00

The average case occurs in the linear search algorithm

a. When the item is the last element in the array
b. When the item is somewhere in the middle of the array ✓
c. Item is the last element in the array or item is not there at all
d. When the item is not the array at all

Question **15**

Correct

Mark 1.00 out of 1.00

Given an array arr = {45,77,89,90,94,99,100} and key = 100; What are the mid values(corresponding array elements) generated in the first and second iterations?

a. 94 and 99

b. 90 and 100

c. 89 and 94

d. 90 and 99 ✓

Status	Finished
Started	Friday, 30 October 2026, 3:49 PM
Completed	Friday, 30 October 2026, 3:59 PM
Duration	9 mins 7 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.

Constraints

1<= string length <= 200

Sample Input 1

experience
enc

Sample Output 1

xpri

Answer: (penalty regime: 0 %)

```
s1 = input()
s2 = input()

s2_set = set(s2)
result = "".join(char for char in s1 if char not in s2_set)

print(result)
```

	Input	Expected	Got	
✓	experience enc	xpri	xpri	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

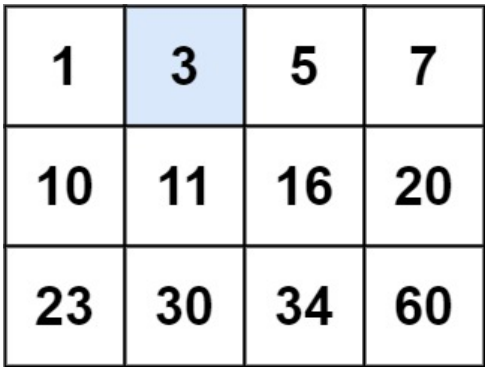
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `True` *if* `target` *is in* `matrix` *or* `False` *otherwise*.

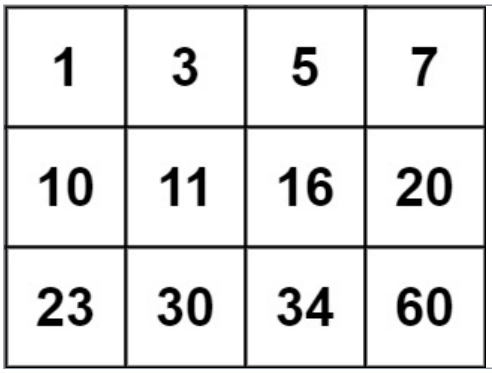
You must write a solution in `O(log(m * n))` time complexity.

Example 1:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: True
```

Example 2:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: False
```

For example:

Test	Result
`print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))`	`False`
`print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))`	`True`

Answer: (penalty regime: 0 %)

Reset answer

```
def searchMatrix(matrix, target):
    if not matrix or not matrix[0]:
        return False

    m, n = len(matrix), len(matrix[0])
```

```
    low, high = 0, (m * n) - 1

    while low <= high:
        mid = (low + high) // 2
        row = mid // n
        col = mid % n
        val = matrix[row][col]

        if val == target:
            return True
        elif val < target:
            low = mid + 1
        else:
            high = mid - 1

    return False

```

	Test	Expected	Got	
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False	False	✓
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Incorrect

Mark 0.00 out of 1.00

String should contain only the words are not palindrome.

Sample Input 1

Malayalam is my mother tongue

Sample Output 1

is my mother tongue

Answer: (penalty regime: 0 %)

```
import sys


def remove_palindromes():
    lines = sys.stdin.read().splitlines()
    for line in lines:
        if not line.strip():
            continue
        words = line.split(" ")
        non_palindromes = [
            w
            for w in words
            if w and (len(w) == 1 or w.lower() != w.lower()[::-1])
        ]
        print(" ".join(non_palindromes))


if __name__ == "__main__":
    remove_palindromes()
```

	Input	Expected	Got	
✓	Malayalam is my mother tongue	is my mother tongue	is my mother tongue	✓

Your code failed one or more hidden tests.
Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

Example 1:

```
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
```

Example 2:

```
Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.
```

Example 3:

```
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.
```

For example:

Test	Result
`print(missingNumber([3,0,1]))`	2
`print(missingNumber([0,1]))`	2

Answer: (penalty regime: 0 %)

Reset answer

```
def missingNumber(nums):
    n = len(nums)
    expected_sum = n * (n + 1) // 2
    actual_sum = sum(nums)
    return expected_sum - actual_sum

```

	Test	Expected	Got	
✓	`print(missingNumber([3,0,1]))`	2	2	✓
✓	`print(missingNumber([0,1]))`	2	2	✓
✓	`print(missingNumber([9,6,4,2,3,5,7,0,1]))`	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

An list contains N numbers and you want to determine whether two of the numbers sum to a given number K. For example, if the input is 8, 4, 1, 6 and K is 10, the answer is yes (4 and 6). A number may be used twice.

Input Format

The first line contains a single integer n , the length of list

The second line contains n space-separated integers, list[i].

The third line contains integer k.

Output Format

Print Yes or No.

Sample Input

7

0 1 2 4 6 5 3

1

Sample Output

Yes

For example:

Input	Result
5 8 9 12 15 3 11	Yes
6 2 9 21 32 43 43 1 4	No

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    nums = list(map(int, input_data[1:n+1]))
    k = int(input_data[n+1])

    seen = set()
    found = False

    for num in nums:
        if (k - num) in seen or (num * 2 == k):
            found = True
            break
        seen.add(num)

    print("Yes" if found else "No")

if __name__ == "__main__":
```

```
24      main()
```

	Input	Expected	Got	
✓	5 8 9 12 15 3 11	Yes	Yes	✓
✓	6 2 9 21 32 43 43 1 4	No	No	✓
✓	6 13 42 31 4 8 9 17	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Balanced strings are those that have an equal quantity of 'L' and 'R' characters.

Given a balanced string s, split it in the maximum amount of balanced strings.

Return the maximum amount of split balanced strings.

Example 1:

Input:

RLRRLLRLRL

Output:

4

Explanation: s can be split into "RL", "RRLL", "RL", "RL", each substring contains same number of 'L' and 'R'.

Example 2:

Input:

RLLLLRRRLR

Output:

3

Explanation: s can be split into "RL", "LLLRRR", "LR", each substring contains same number of 'L' and 'R'.

Example 3:

Input:

LLLLRRRR

Output:

1

Explanation: s can be split into "LLLLRRRR".

Constraints:

1 <= s.length <= 1000

s[i] is either 'L' or 'R'.

s is a balanced string.

For example:

Test	Result
`print(BalancedStrings('RLRRLLRLRL'))`	4
`print(BalancedStrings('RLLLLRRRLR'))`	3

Answer: (penalty regime: 0 %)

Reset answer

```
def BalancedStrings(s):
    count = 0
    balance = 0

    for char in s:
        if char == "R":
            balance += 1
        elif char == "L":
            balance -= 1

```

```
        if balance == 0:
            count += 1

    return count
```

	Test	Expected	Got	
✓	print(BalancedStrings('RLRRLLRLRL'))	4	4	✓
✓	print(BalancedStrings('RLLLLRRRLR'))	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

Example 2:

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 < nums[i], target < 10^4`
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

For example:

Test	Result
`print(search([-1,0,3,5,9,12],9))`	4

Answer: (penalty regime: 0 %)

Reset answer

```
def search(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

	Test	Expected	Got	
✓	`print(search([-1,0,3,5,9,12],9))`	`4`	`4`	✓
✓	`print(search([-1,0,3,5,9,12],2))`	`-1`	`-1`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given an list, find peak element in it. A peak element is an element that is greater than its neighbors.

An element a[i] is a peak element if

A[i-1] <= A[i] >=a[i+1] for middle elements. [0<i<n-1]

A[i-1] <= A[i] for last element [i=n-1]

A[i]>=A[i+1] for first element [i=0]

Input Format

The first line contains a single integer n , the length of A .
The second line contains n space-separated integers,A[i].

Output Format

Print peak numbers separated by space.

Sample Input

5

8 9 10 2 6

Sample Output

```
10 6
```

For example:

Input	Result
4 12 3 6 8	12 8

Answer: (penalty regime: 0 %)

```
import sys

def find_peak_elements():
    # Read all inputs from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    arr = list(map(int, input_data[1:n+1]))

    peaks = []

    # Handle edge case where array has only 1 element
    if n == 1:
        print(arr[0])
        return

    # Check each position against boundary and neighbor rules
    for i in range(n):
        if i == 0:
            # First element: only compare with right neighbor
            if arr[0] >= arr[1]:
                peaks.append(arr[0])
        elif i == n - 1:
            # Last element: only compare with left neighbor
            if arr[n - 1] >= arr[n - 2]:
                peaks.append(arr[n - 1])
```

```
                peaks.append(arr[i - 1])
        else:
            # Middle elements: must be >= both left and right neighbors
            if arr[i] >= arr[i - 1] and arr[i] >= arr[i + 1]:
                peaks.append(arr[i])

    # Print peaks as space-separated values
    print(*(peaks))

if __name__ == "__main__":
    find_peak_elements()
```

	Input	Expected	Got	
✓	7 15 7 10 8 9 4 6	15 10 9 6	15 10 9 6	✓
✓	4 12 3 6 8	12 8	12 8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Two string values S1, S2 are passed as the input. The program must print first N characters present in S1 which are also present in S2.

Input Format:

The first line contains S1.
The second line contains S2.
The third line contains N.

Output Format:

The first line contains the N characters present in S1 which are also present in S2.

Boundary Conditions:

2 <= N <= 10
2 <= Length of S1, S2 <= 1000

Example Input/Output 1:

Input:

abcbde
cdefghbb
3

Output:

bcd

Note:

b occurs twice in common but must be printed only once.

Answer: (penalty regime: 0 %)

```
import sys

def print_common_characters():
    # Read all inputs from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    s1 = input_data[0]
    s2 = input_data[1]
    n = int(input_data[2])

    # Store s2 characters in a hash set for O(1) lookup
    s2_set = set(s2)
```

```
    s2_set = set(s2)

    result = []
    seen = set()

    # Iterate through S1, maintaining relative order
    for char in s1:
        if char in s2_set and char not in seen:
            seen.add(char)
            result.append(char)
            if len(result) == n:
                break

    print("".join(result))

if __name__ == "__main__":
    print_common_characters()
```

	Input	Expected	Got	
✓	abcbde cdefghbb 3	bcd	bcd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a Python program for binary search.

For example:

Input	Result
1,2,3,5,8 6	False
3,5,9,45,42 42	True

Answer: (penalty regime: 0 %)

```
import sys

def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return True
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return False

def main():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Parse comma-separated list of integers
    arr = list(map(int, input_data[0].strip().split(',')))
    target = int(input_data[1].strip())

    # Ensure list is sorted for binary search
    arr.sort()

    # Perform binary search
    result = binary_search(arr, target)
    print(result)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	1,2,3,5,8 6	False	False	✓
✓	3,5,9,45,42 42	True	True	✓
✓	52,45,89,43,11 11	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.